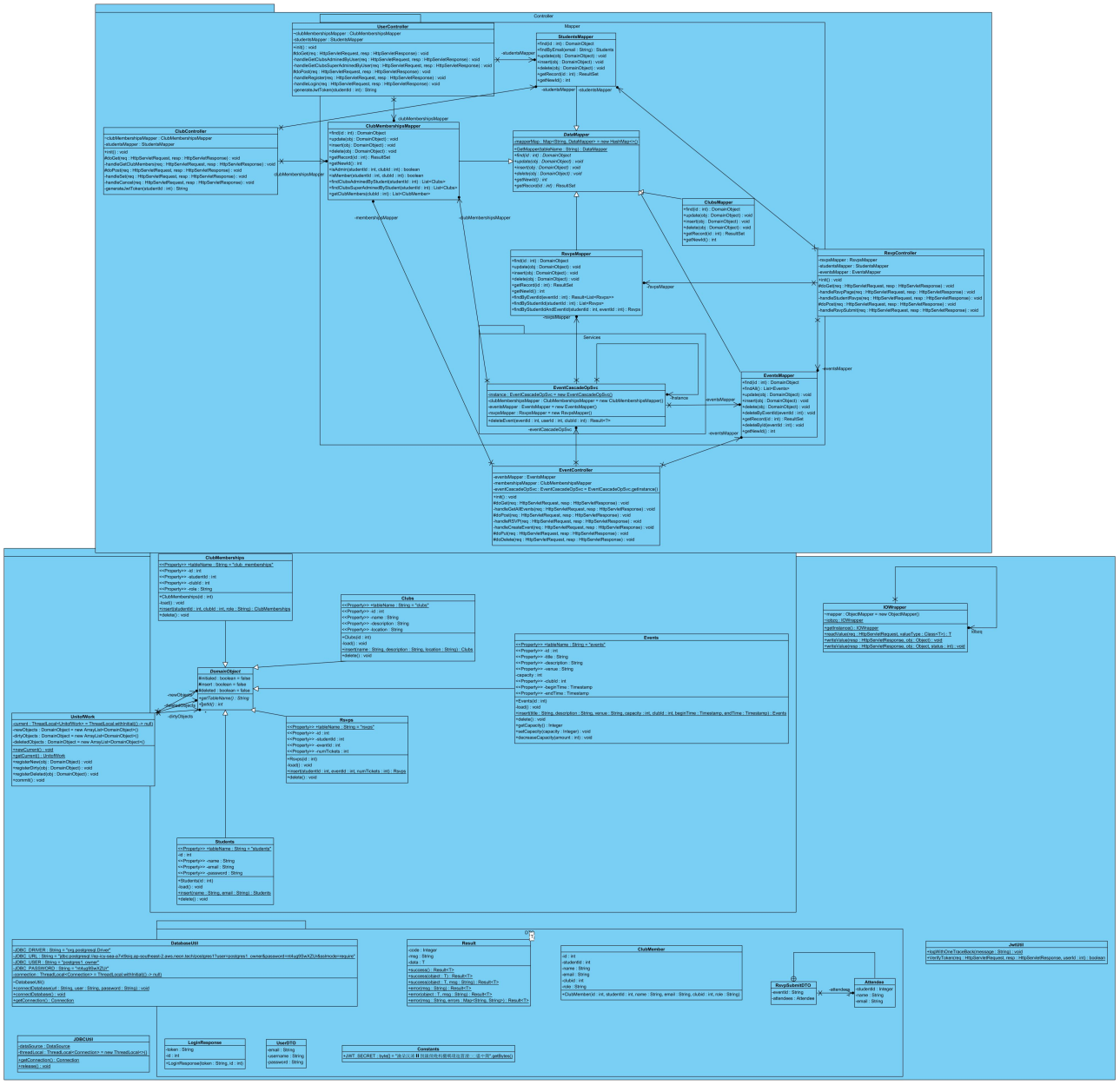
SWEN 90007 24S2 Assignment Part 2 Report

Rong Wang, Pengyuan Yu, Xiang Wang, Yongchun Li

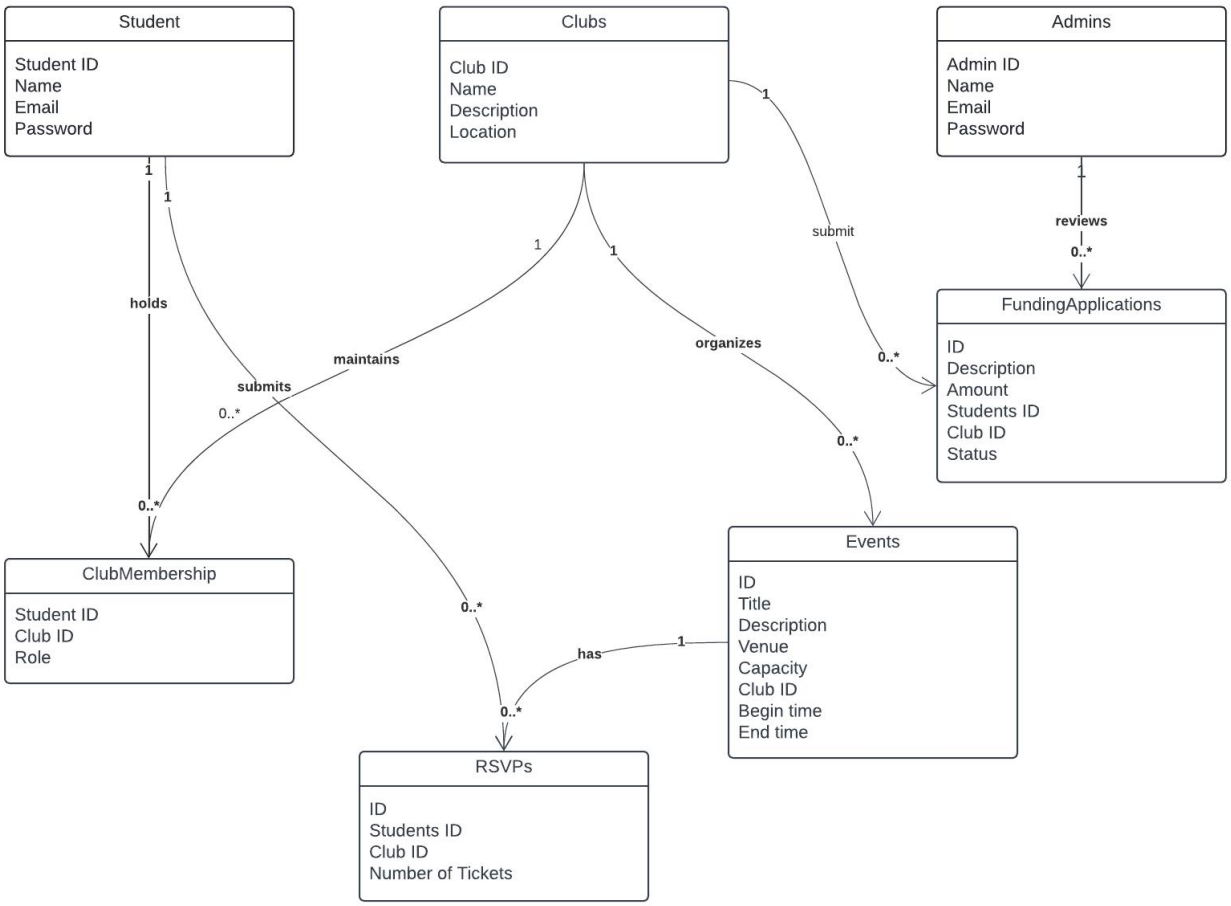
Chapter One General Architecture

1.1 Class Diagram



Chapter Two implemented patterns

2.1 Domain model



2.1.1 Entities

2.1.1.1 Student

Represents students who participate in clubs.
Attributes: Student ID, Name, Email, Password.
Relationship: A Student can hold multiple Club Memberships (one-to-many).

2.1.1.2 Clubs

Represents a university club.
Attributes: Club ID, Name, Description, Location.
Relationship: Each Club is maintained by multiple Club Memberships and organizes Events.

2.1.1.3 Admins

Represents administrators who have access to dashboard to review funding applications.

Attributes: Admin ID, Name, Email, Password.
Relationship: Admins review Funding Applications submitted by Students.

2.1.1.4 Club Membership

Defines the relationship between Students and Clubs. A Student can be a Member or Admin of multiple Clubs (many-to-many relationship).
Attributes: Student ID, Club ID, Role.

2.1.1.5 Events

Represents events organized by Clubs.
Attributes: ID, Title, Description, Venue, Capacity, Club ID, Begin time, End time.
Relationship: A Club can organize many Events, and students can RSVP to attend Events (many-to-many relationship through RSVPs).

2.1.1.6 RSVPs

Represents a student's attendance or registration for a specific event.
Attributes: ID, Students ID, Club ID, Number of Tickets.
Relationship: A Student can have multiple RSVPs for different Events.

2.1.1.7 Funding Applications:

Represents applications for funding submitted by students for their clubs.
Attributes: ID, Description, Amount, Student ID, Club ID, Status.
Relationship: Reviewed by Admins and linked to both Students and Clubs.

2.1.2 Relationships:

Student → Club Membership: A Student can hold memberships in multiple Clubs, represented by the ClubMembership entity.
Club → Event: A Club can organize multiple Events.
Student → RSVP → Event: A Student can RSVP to attend multiple Events.
Admin → Funding Applications: Admins review Funding Applications submitted by students.

2.2 Data mapper

Data mappers serve as a layer between the application's business logic and the database. Their primary role is to transfer data between the database and the application's in-memory objects without the objects needing to know anything about the database access code. This separation ensures that the business logic can focus on what the application needs to do, not how the data is stored or retrieved. Each data mapper corresponds to a specific entity in thet domain model.

2.2.1 StudentMapper

The StudentMapper is responsible for managing all interactions between the Student entity and the students table in the database. This mapper handles the translation of Student objects to database

rows and vice versa. Each student’ s information, such as ID, name, email, and password, is stored and managed through this mapper.

Functions:
findById(int id): Retrieves a specific student from the database using their unique ID. This function queries the students table and maps the result to a Student object.
insert(Student student): Inserts a new student into the database. The method takes a Student object and maps its attributes (name, email, password) to the corresponding fields in the students table.
update(Student student): Updates an existing student's information in the database. If, for example, a student changes their email or password, this method ensures those changes are reflected in the students table.
delete(int id): Deletes a student from the database by their ID. This removes the corresponding row in the students table.

2.2.2 ClubMapper

The ClubMapper manages all interactions between the Club entity and the clubs table. Each club, including its name, description, and location, is stored in the database, and this mapper translates between club objects and database records.

Functions:
findById(int id): Finds a specific club based on its unique club ID. This method queries the clubs table and returns a corresponding Club object.
insert(Club club): Inserts a new club into the database. This takes the club's attributes, such as the club name, description, and location, and maps them into the clubs table.
update(Club club): Updates the details of an existing club in the database. If, for example, a club changes its location or description, this method ensures that those updates are made.
delete(int id): Deletes a club from the database based on its ID. This removes the club record from the clubs table.

2.2.3 EventMapper

The EventMapper handles the mapping between the Event entity and the events table. Events organized by clubs are stored in the database, including details like the event's title, venue, date, and capacity.

Functions:

findById(int id): Finds an event by its unique event ID, retrieving the event’ s details from the events table and mapping them to an Event object.
insert(Event event): Inserts a new event into the database. This function maps the event’s attributes, such as title, venue, capacity, and date, to the appropriate fields in the events table.

update(Event event): Updates an existing event in the database. This method modifies event details, like updating the venue or changing the event date, in the events table.
delete(int id): Deletes an event from the database by its ID, removing the event's record from the events table.

2.2.4 RSVPMapper

The RSVPMapper is responsible for managing the relationship between students and events, specifically tracking which students have RSVP’d to which events. This mapper interacts with the rsvps table, which stores the RSVP details such as the student’s ID, event ID, and number of tickets.

Functions:
findByEventId(int eventId): Retrieves a list of students who have RSVP'd for a specific event based on the event ID. This function queries the rsvps table for all rows matching the event ID and maps them to a list of RSVP objects.
findByStudentId(int studentId): Retrieves a list of events a specific student has RSVP’d to, based on the student ID. This function queries the rsvps table and maps the results to RSVP objects.
insert(RSVP rsvp): Inserts a new RSVP into the rsvps table. This method maps the RSVP details (student ID, event ID, number of tickets) to the appropriate fields in the database.
delete(int rsvpId): Deletes an RSVP record based on its ID. This removes the RSVP from the rsvps table, meaning that the student will no longer be counted as attending the event.

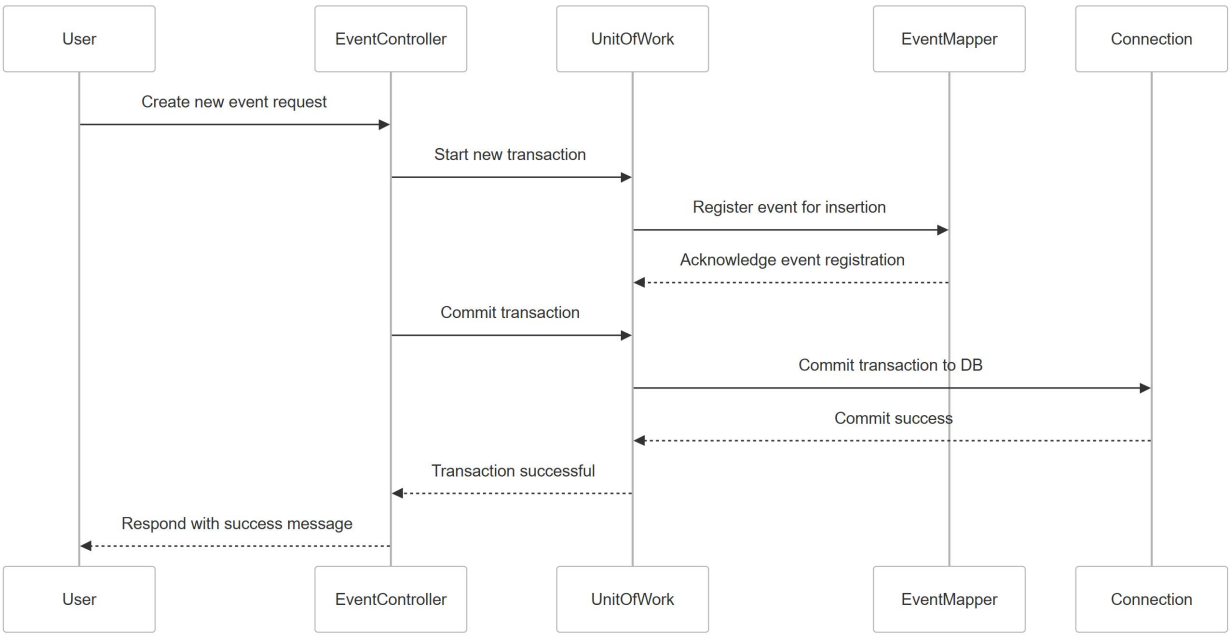
2.2.5 ClubMembershipMapper

The ClubMembershipMapper handles the relationship between students and clubs, tracking which students belong to which clubs. It works with the club_memberships table, which stores the student’s ID, club ID, and their role within the club (e.g., member, president).

Functions:
findByClubId(int clubId): Retrieves a list of students who are members of a specific club based on the club ID. The method queries the club_memberships table and maps the results to ClubMembership objects.
findByStudentId(int studentId): Retrieves a list of clubs a student belongs to, based on their student ID. This function queries the club_memberships table and maps the results to a list of ClubMembership objects.
insert(ClubMembership clubMembership): Adds a new club membership into the database, mapping the membership details (student ID, club ID, role) to the appropriate fields in the club_memberships table.
delete(int membershipId): Removes a student’s membership from a club by deleting the corresponding record from the club_memberships table.

2.3 Unit of work

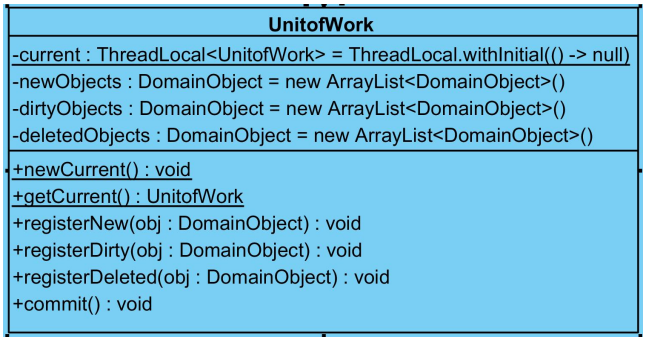
2.3.1 Implementation and usage



In our System, various operations such as creating, updating, or deleting events, RSVPs, and funding applications often involve multiple database actions that need to be treated as a single atomic unit. Without a structured approach, handling these operations independently can result in data inconsistencies, particularly when failures occur partway through an operation sequence.

Therefore, the Unit of Work pattern is implemented to bundle multiple database operations into a single transaction. This guarantees consistency by ensuring either all operations succeed, or none do, rolling back the changes if necessary. This pattern allows for centralized transaction management, making the system more reliable when dealing with complex operations.

The class UnitOfWork manages new, dirty (modified), and deleted objects, coordinating their persistence in the database by registering them. It maintains a thread-local instance for transaction boundaries.



Usage in Controllers: The UnitofWork class is integrated into the controllers (e.g., EventController). When a request to create, update, or delete an event is processed, all relevant operations are registered with the UnitofWork instance. Upon successful execution, the changes are committed to the database. If any failure occurs, the changes are rolled back.

For example, in the EventController, we have 70+ lines of code wrapped by UnitofWork.*newCurrent*() and UnitofWork.*getCurrent*().commit();

2.3.2 Advantages

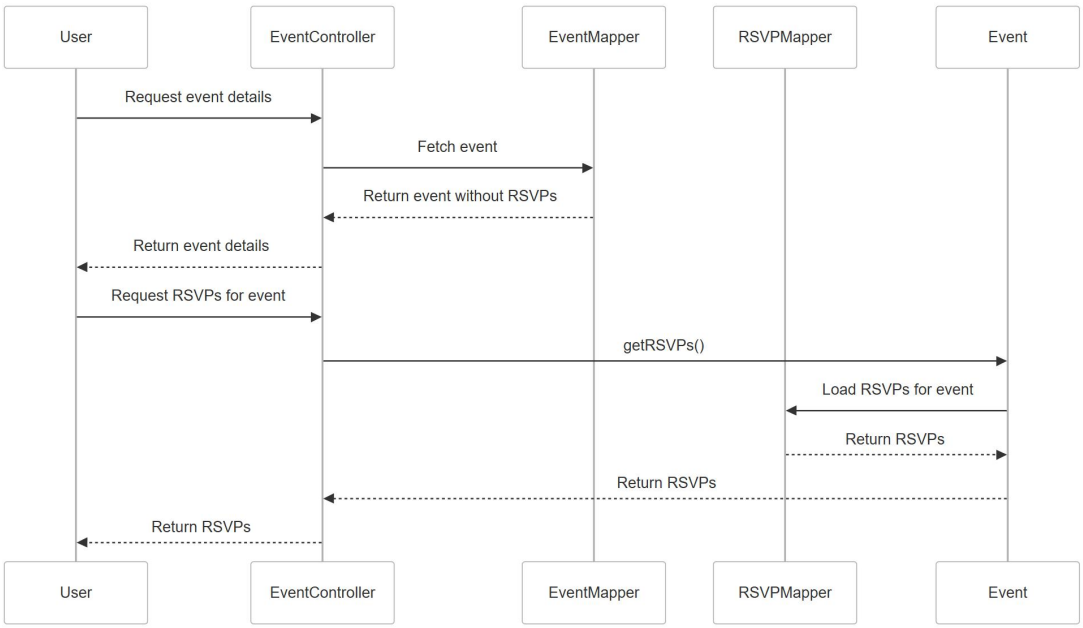
Consistency: Ensures that all changes within a transaction are either fully committed or rolled back, maintaining data integrity.

Centralized Management: All transaction logic is encapsulated within the UnitofWork class, simplifying management and maintenance.

2.3.3 Alternatives Considered

Direct Transaction Management: This alternative would involve managing transactions directly in each mapper or service. However, it would scatter the transaction logic throughout the system, making it harder to maintain and extend. The Unit of Work pattern provides better cohesion and reduces complexity.

2.4 Lazy load



Loading all related data (e.g., RSVPs for events) immediately can be inefficient, especially when this data is not always required. In the University Club Management System, loading unnecessary data can degrade performance and increase memory usage.

The Lazy Load pattern is implemented to defer the loading of associated data until it is actually needed. This pattern is applied, for instance, in loading RSVP details for an event. Instead of retrieving all RSVPs when loading an event, they are only fetched when the getRSVPs() method is invoked. Another example is the obtain of entity class(i.e. DomainObject), In getRecord(id) of mappers, a series of result sets are obtained from the database based on the id, but only when used, the corresponding attributes are injected into the data object and the corresponding operations are performed.

2.4.1 Alternatives Considered

Eager Loading: This alternative would load all related entities upfront, but it would lead to performance bottlenecks when dealing with large datasets or unnecessary information. Lazy loading is more suitable for this use case because most of the related data is not always needed.

2.5 Identity field

Database uses `SERIAL` ID for every table, and stores the ID in the object. When creating a new object, which requires a new ID, the server queries the database for a new ID to ensure the primary key constraint.

2.6 Foreign key mapping

For foreign key mapping, the object contains all fields of a table, with one object corresponding to one table. Therefore, it also includes the foreign key mapping.

2.7 Association table mapping

Association Table Mapping is a common pattern used in databases to represent many-to-many relationships between entities. Since relational databases cannot directly store many-to-many relationships, an association table (also called a junction table or join table) is used to break this into two one-to-many relationships. In our project, we use ClubMembership to connect students with clubs and RSVPs to connect students with events they RSVP to. These association tables store foreign keys from both related tables (e.g., Student ID and Club ID in ClubMembership) to create the association.

2.8 One of the inheritance patterns

2.8.1 DataMapper as the Base Class

The DataMapper serves as a generic class that implements common behavior for interacting with the database. It handles generic CRUD operations (Create, Read, Update, Delete) and database connections, abstracting away the low-level details of data persistence. Each specific entity, like Club, Event, Student, etc., has its own dedicated mapper (e.g., ClubMapper, EventMapper), which inherits from DataMapper to reuse this behavior.

Common Features of the DataMapper:

CRUD Operations: The DataMapper class typically includes methods like insert(), update(), delete(), and findById(). These methods operate on generic data models but are designed in a way that the child classes can easily customize them.

Connection Management: The base class might handle database connection logic, ensuring that the connection is opened and closed appropriately.

Query Execution: There are methods in DataMapper for executing SQL queries and possibly for mapping result sets to objects.

Generic Handling: The DataMapper might also include generic error handling, query preparation, and database transaction management.

2.8.2 Specific Mappers Extending DataMapper

Each specific mapper (like ClubMapper, EventMapper, etc.) extends DataMapper, inheriting the common functionality and defining entity-specific logic. These mappers can:

Reuse inherited methods: Methods like insert(), update(), or delete() from DataMapper can be used directly without modification in many cases, as they handle the database operations.
Override inherited methods: If an entity requires custom behavior (e.g., special validation, complex queries), the specific mapper can override the method inherited from DataMapper.
Add custom methods: Each mapper may also introduce new methods specific to the entity. For instance, EventMapper may add a method to retrieve all upcoming events, while ClubsMapper may add a method to find clubs by a particular attribute.

2.9 Authentication and Authorization

Once a user login is successfully authenticated, the server encrypts the user ID into a JWT (JSON Web Token) and attaches it to the LoginResponseDTO. This token is then stored locally in the browser with an expiration time—once expired, any subsequent request will require re-authentication. During requests, the token must be inserted into the request header. This approach helps mitigate impersonation while strengthening the validation of request privileges. However, JWT does not guard against man-in-the-middle attacks. We plan to integrate Spring Security in the next sprint to address this vulnerability.